

Redes y Comunicaciones

Ing. Mariano Coccia

Instituto Superior Juan XXIII

Material de apoyo del texto Computer Networking:
A Top Down Approach Featuring the Internet.

Capítulo 3: Capa Transporte

Contenido de Capítulo 3

3.1 Servicios de la capa transporte

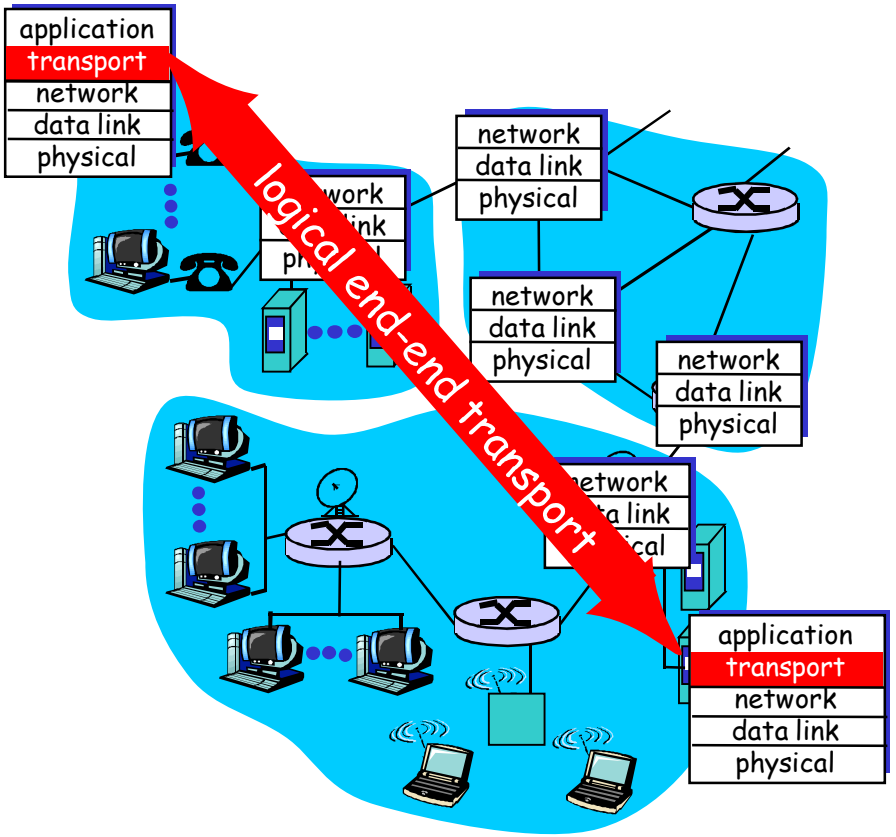
3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

Protocolos y servicios de transporte

- Proveer *comunicación lógica* entre procesos de aplicación corriendo en diferentes hosts
- Los protocolos de transporte corren en sistemas terminales (computadores, no equipos internos como routers)
- Hay más de un protocolo de transporte disponible para las aplicaciones
  - Más comunes: TCP y UDP
  - Hoy también SCTP (Stream Control Transmission Protocol, 2000),
    DCCP (Datagram Congestion Control Protocol, 2005)



Capa transporte vs. capa red

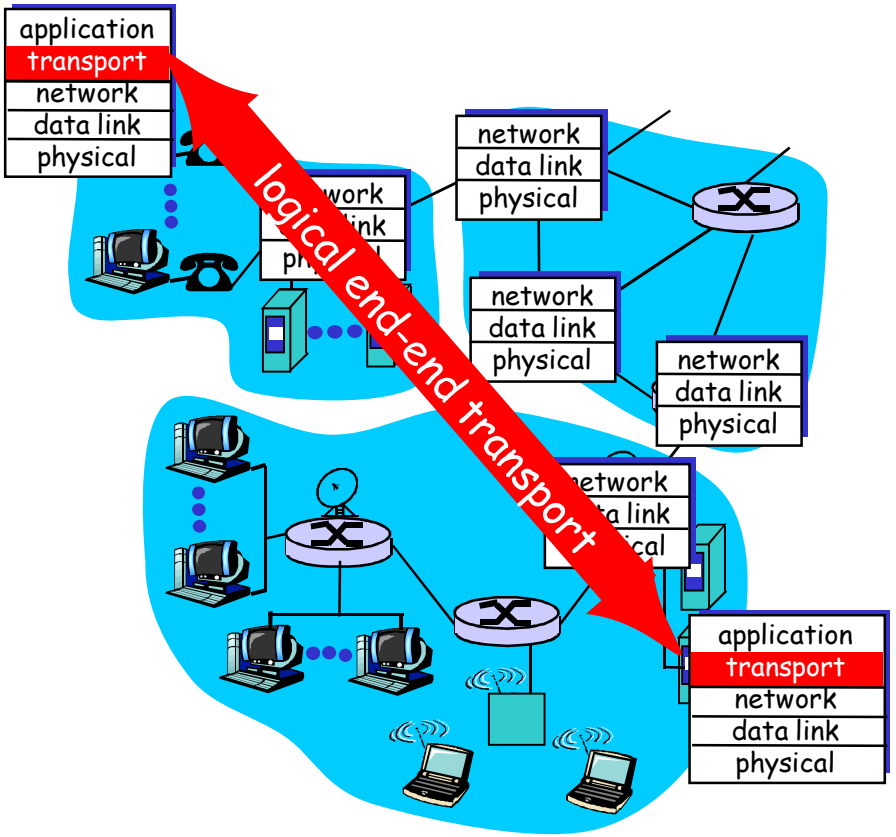
- *Capa transporte:* encargada de la comunicación lógica entre procesos
  - Depende y mejora los servicios de la capa de red
- *Capa de red:* encargada de la comunicación lógica entre hosts

Analogía:

- *12 niños en una casa envían cartas a 12 niños. en otra casa*
- *Ann y Bill recopilan las cartas en cada hogar y las envían por correo. También distribuyen las cartas que llegan.*
- Procesos = niños
- Mensajes aplicación = cartas en sobres
- Hosts = casas
- Protocolo de transporte = Ann y Bill
- Protocolo capa red = servicio de correos

Protocolos de capa transporte en Internet

- TCP: Entrega confiable y en orden
  - Tiene: Control de flujo
  - Tiene: control de congestión
  - Establecimiento de conexión
- UDP: Entrega no confiable, tal vez desordenada:
  - Básicamente el mismo servicio de IP: "mejor esfuerzo" (best-effort)
- Qué servicios no se ofrecen:
  - Garantías de retardo
  - Garantías de ancho de banda
  - (Básicamente porque no es fácil -o posible- implementarlo basándose en los servicios de IP)



Contenido de Capítulo 3

3.1 Servicios de la capa transporte

3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

UDP: User Datagram Protocol [RFC 768]

- Básicamente agrega puerto a paquete IP
- Servicio de "mejor esfuerzo", un segmento UDP se puede:
  - perder
  - Entregar a la aplicación fuera de orden
- *Sin conexión:*
  - No hay handshaking (establecimiento de conexión) entre servidor y receptor UDP
  - Cada segmento UDP es manejado en forma independiente de los otros

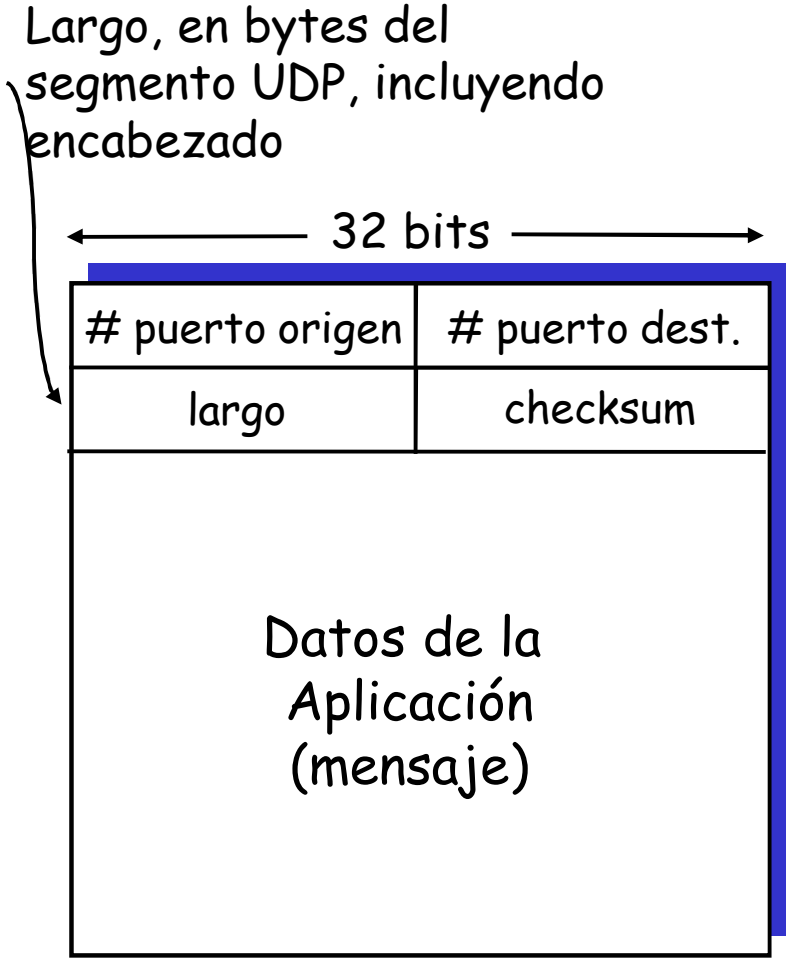
"Mejor esfuerzo": Haré todo lo posible por entregarlo, pero no lo garantizo.

¿Por qué existe UDP?

- No requiere establecimiento de conexión (lo cual agregaría retardo)
- Simple: no se requiere mantener estado en el Tx y el Rx
- Pequeño segmento de encabezado => menor overhead
- No hay control de congestión: UDP puede transmitir tan rápido como se desee

UDP: más detalle

- A menudo es usado por flujos (streaming) multimedia en aplicaciones que:
  - Toleran pérdida de datos
  - Son sensibles a la tasa de transmisión
- Otros usos de UDP
  - DNS
  - SNMP (Simple Network Management Protocol)
- Es posible lograr transferencias confiables sobre UDP: se debe programar la confiabilidad en la capa aplicación
  - En este caso la recuperación de errores depende de la aplicación!



Formato segmento UDP

Checksum UDP (suma de chequeo)

Objetivo: detectar "errores" (e.g., bits cambiados) en segmentos transmitidos

Transmisor:

- Trata el contenido de cada segmento como una secuencia de enteros de 16 bits
- checksum: "suma" del contenido del segmento y luego invierte cada bits (se conoce como tomar el complemento 1).
- Transmisor pone el valor del checksum en el campo checksum del datagrama UDP

Receptor:

- Calcula el checksum del segmento recibido
- Chequea si el checksum calculado corresponde al valor de checksum recibido en el campo:
  - NO corresponde => error detectado
  - SI => no hay error detectado. *Pero podrían haber errores sin embargo!*

Contenido de Capítulo 3

3.1 Servicios de la capa transporte

3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

TCP: Generalidades RFCs: 793, 1122, 1323, 2018, 2581

Es una comunicación Punto-a punto:

Un Tx y un Rx

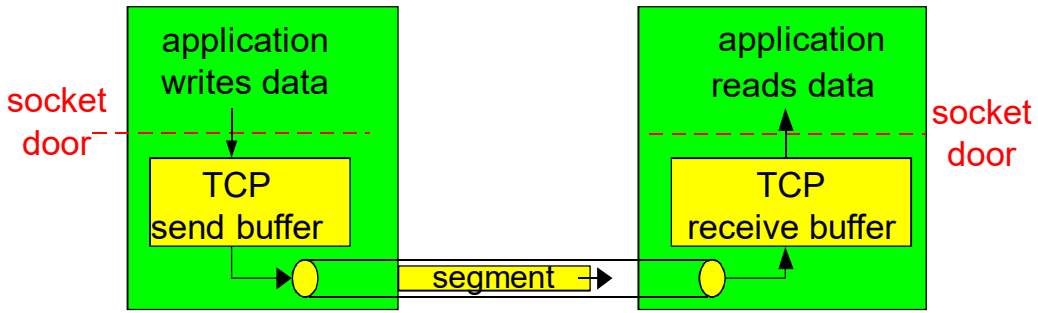
flujo de bytes confiable y en orden:

No hay "límites del inicio/término de mensaje"

Usa pipeline:

El tamaño de la ventana TCP es definido por el control de congestión y control de flujo

Usa buffer en Tx & Rx



- **Transferencia full duplex (dos sentidos):**
  - Flujo de datos bi-direccionales en la misma conexión
  - MSS: maximum segment size, depende del Maximum Transmission Unit de la capa enlace
- **Orientado a la conexión:**
  - Handshaking (intercambio de mensajes de control) inicializa al Tx y Rx antes del intercambio de datos
- **Tiene control de flujo:**
  - Tx no sobrecargará al Rx
- También **tiene control de congestión**

Contenido de Capítulo 3

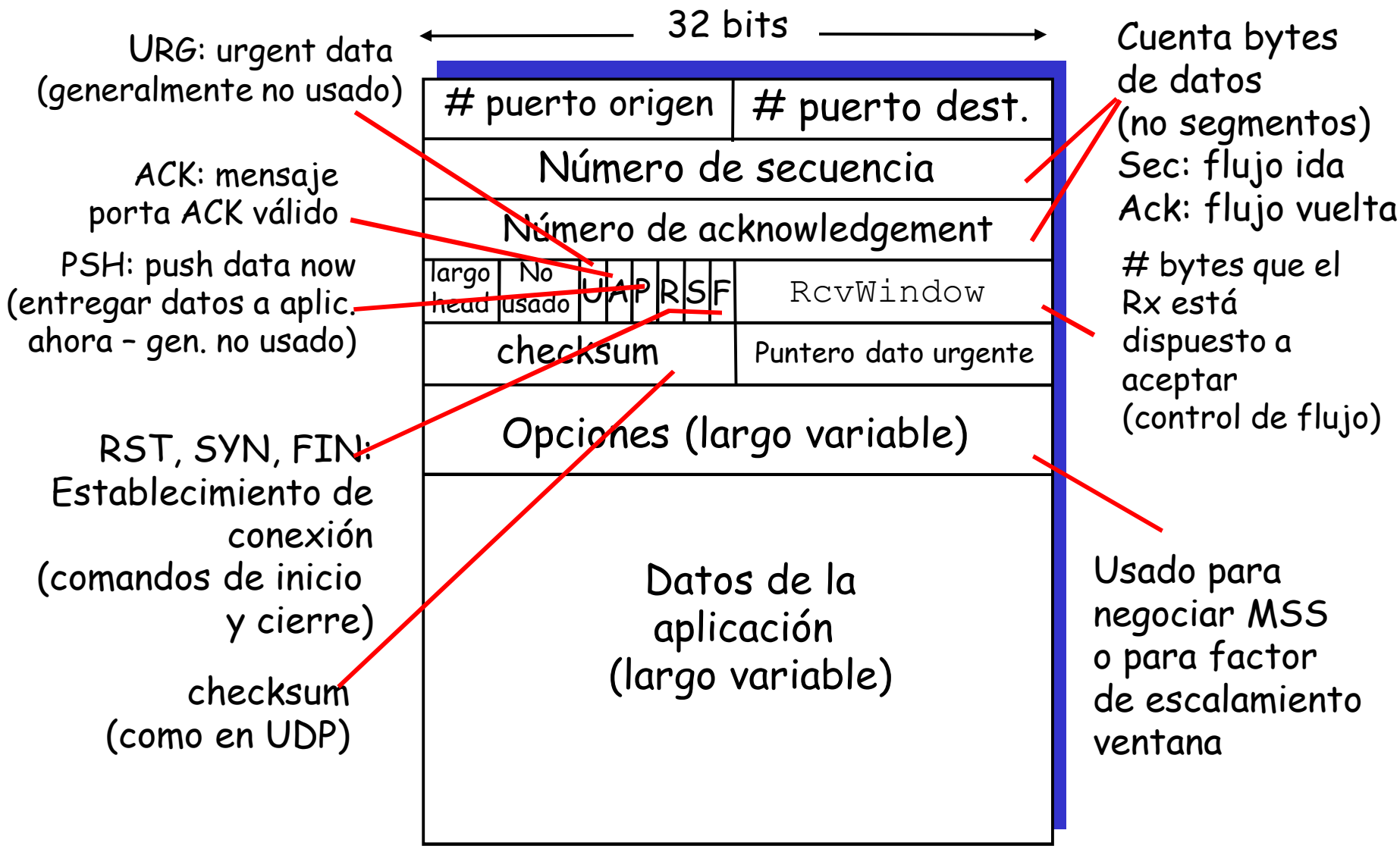
3.1 Servicios de la capa transporte

3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

Estructura de un segmento TCP



Contenido de Capítulo 3

3.1 Servicios de la capa transporte

3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

Transferencia confiable de datos en TCP

- TCP crea un servicio de transferencia confiable sobre el servicio no confiable de IP
- Usa envío de segmentos en pipeline
- ACKs acumulativos
- TCP usa un timer único de retransmisión

Retransmisiones son activadas por:

- Eventos de timeout
- ACKs duplicados

Inicialmente consideremos un Tx TCP simplificado:

- Ignora ACKs duplicados
- Ignora control de flujo y control de congestión

Eventos del transmisor en TCP:

Datos recibidos desde aplicación:

- Crea segmento con # de sec.
- # sec es el número dentro del flujo para el primer byte del segmento
- Inicia timer si no está ya corriendo (asociar el timer al segmento más viejo sin acuse de recibo)
- Tiempo de expiración: TimeOutInterval

Timeout:

Retransmitir el segmento que causó el timeout (sólo 1)

Típicamente el intervalo del timeout se duplica en retransmisiones. ¿Por qué?

Re-iniciar el timer

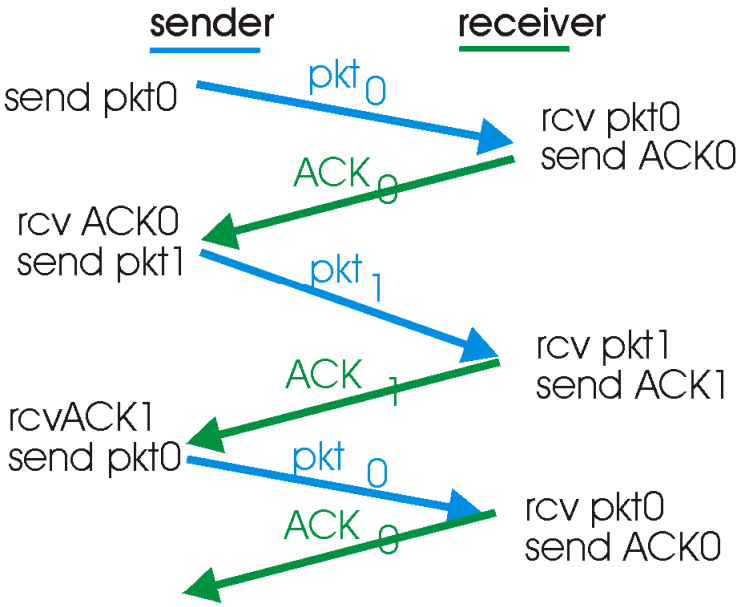
Recepción de ACK:

Si es el ACK de un segmento previo sin acuse de recibo

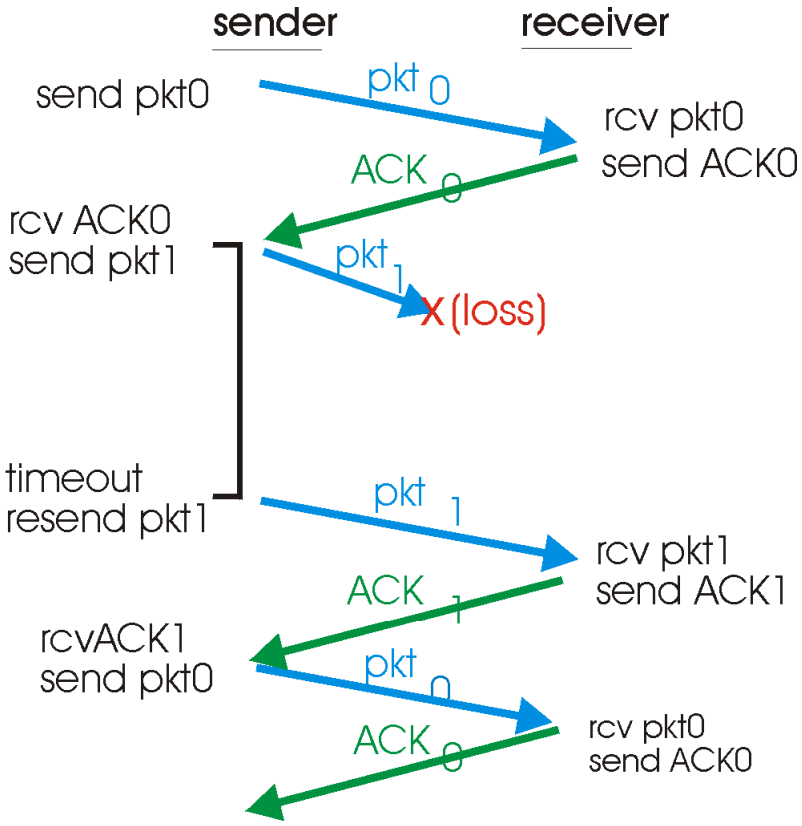
Actualizar estado sobre acuses recibidos

Iniciar timer si hay segmentos sin acuses de recibo.

TCP escenarios (más)

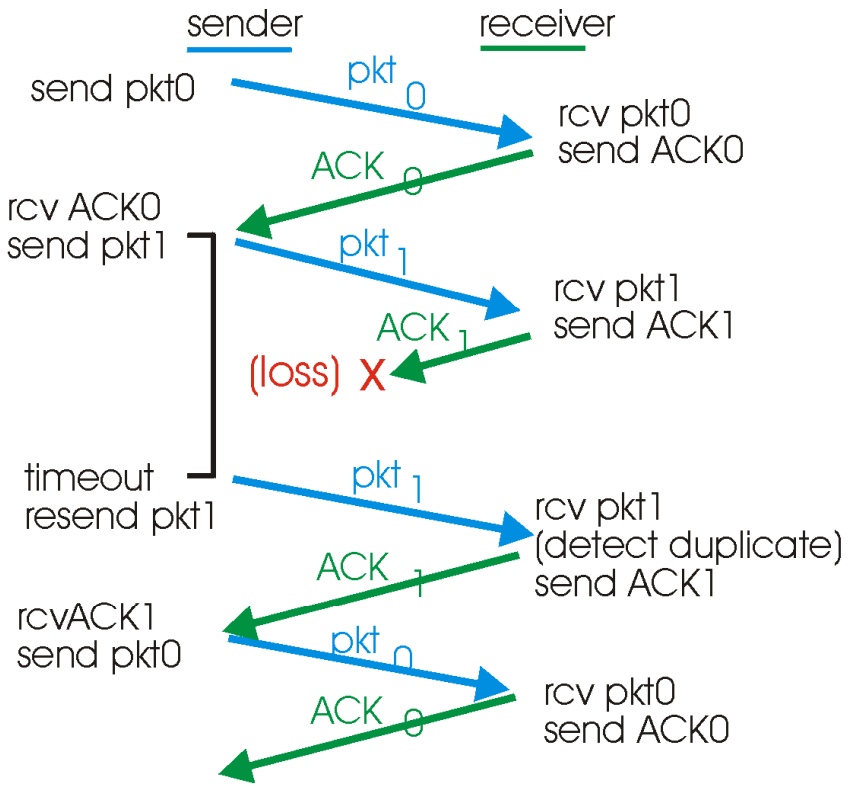


a) Operación **sin** pérdidas

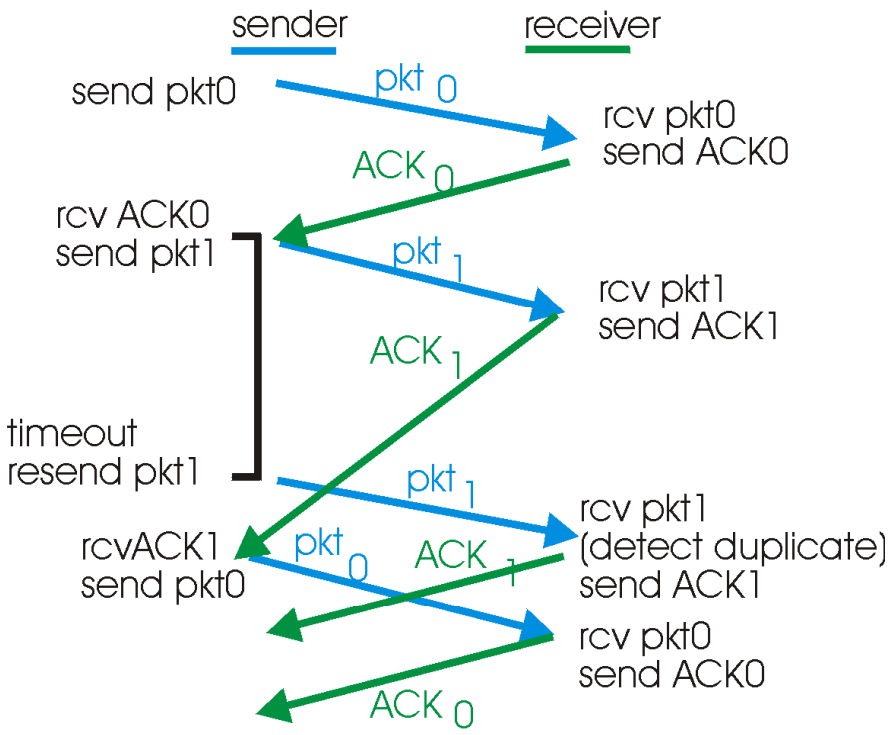


b) Operación **con** pérdidas

TCP escenarios (más)

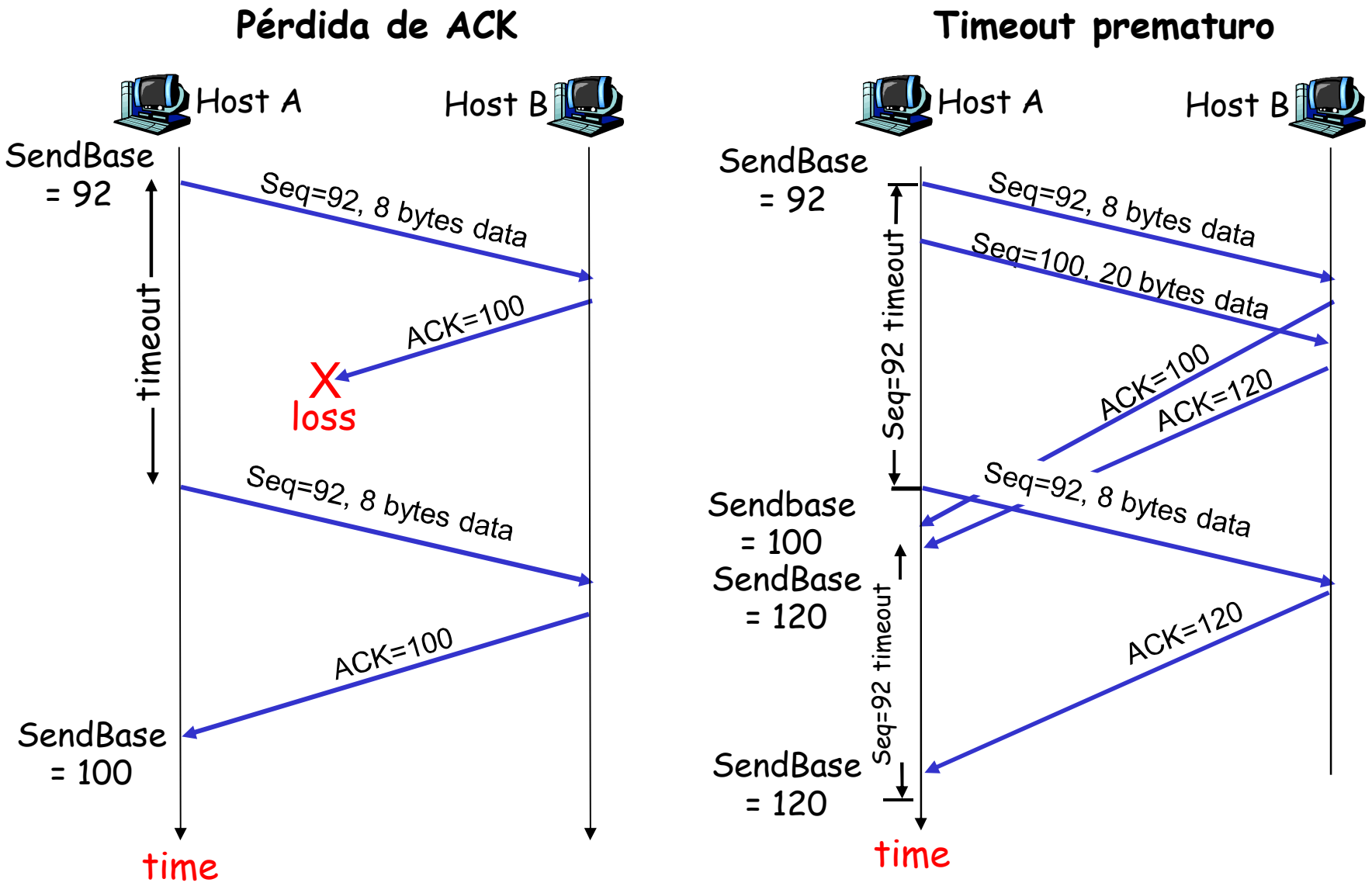


c) Pérdida de ACK

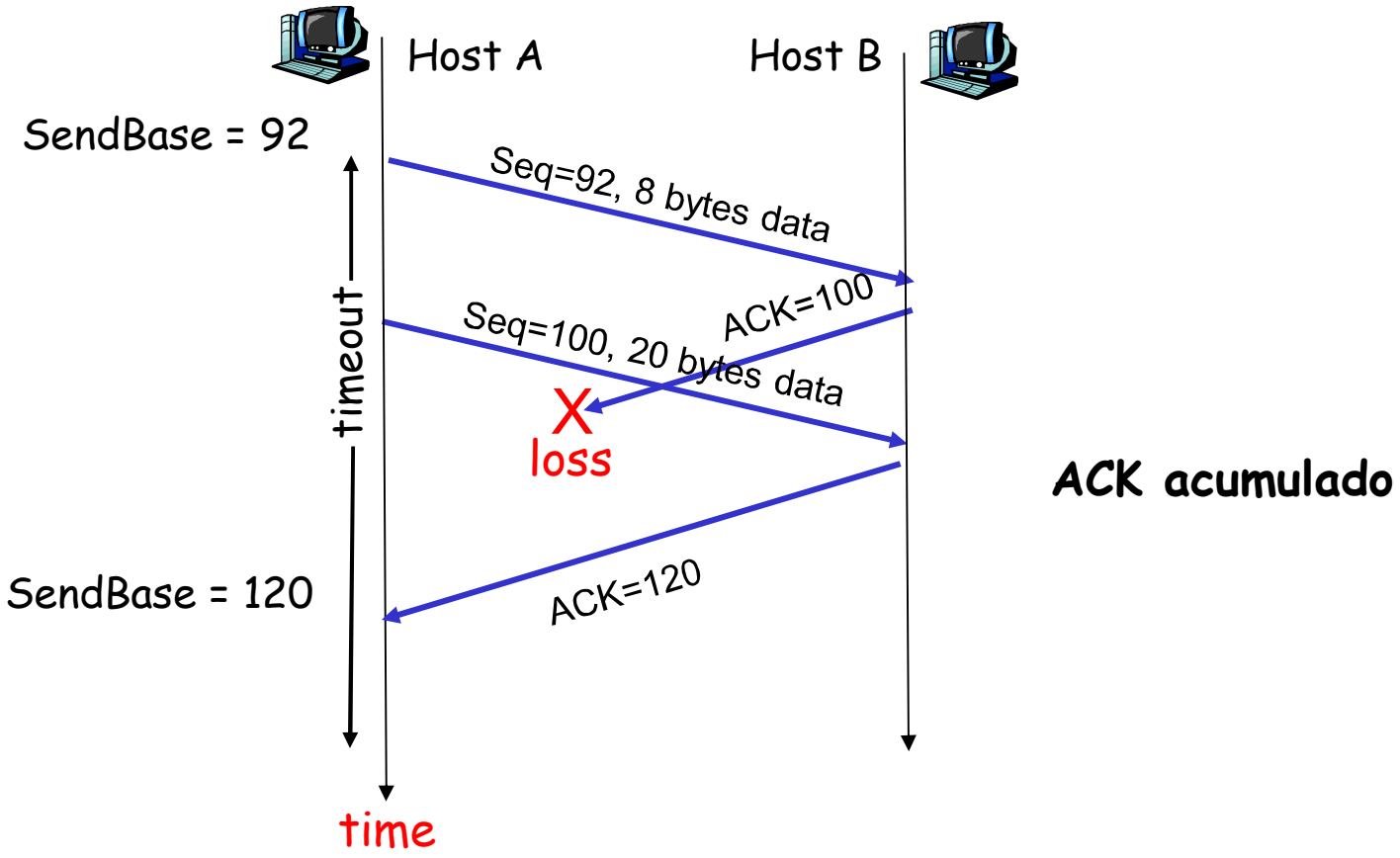


d) Timeout prematuro

TCP: escenarios de retransmisión



TCP escenarios (más)



Estos diagramas no reflejan tiempos de transmisión ni almacenamientos y reenvíos en la ruta.

Contenido de Capítulo 3

3.1 Servicios de la capa transporte

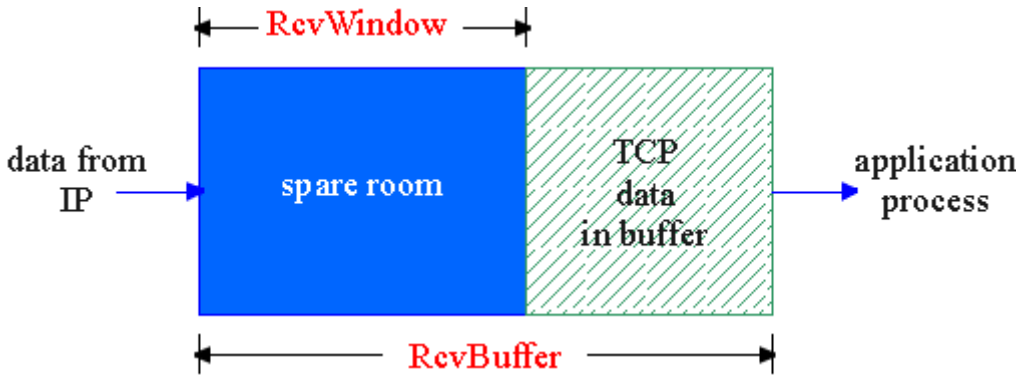
3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

Control de flujo en TCP

- Hemos visto cómo TCP asegura **confiabilidad** en la transferencia, ahora veremos cómo consigue controlar el **flujo de datos**.
- **El lado receptor (Rx) de TCP tiene un buffer receptor de datos:**



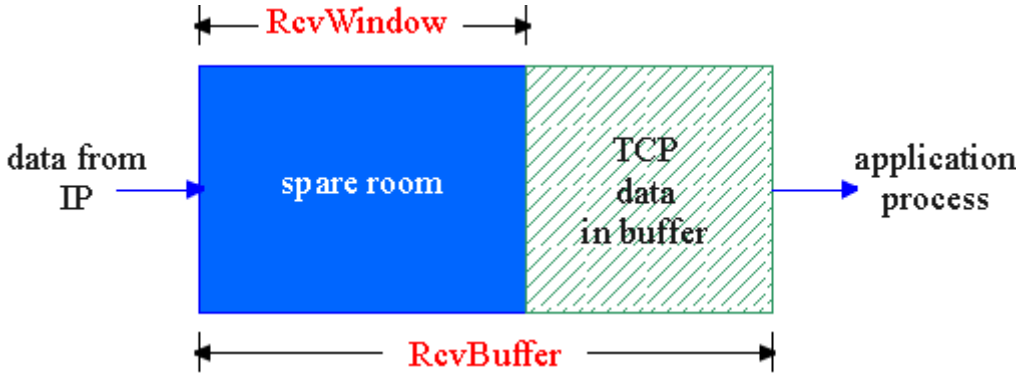
El proceso aplicación puede ser lento en la lectura del buffer (capa transporte).

Control de flujo

Tx no debe sobrecargar el buffer del receptor por transmitir demasiado rápido

- La idea es hacer coincidir la tasa de transmisión con la tasa de lectura de la aplicación.

Control de flujo en TCP: Cómo trabaja



(supongamos que receptor descarta segmentos fuera de orden)

- Espacio libre en buffer

RcvWindow = RcvBuffer - [LastByteRcvd - LastByteRead]

Rx comunica el espacio libre a través del valor de RcvWindow en los segmentos

Así el receptor limita datos en transito (sin ACK) a RcvWindow (Tx debe respetar el no envío de más datos que RcvWindow)

Esto garantiza que el buffer del Rx no se rebalse (overflow)

Control de flujo en TCP: Cómo trabaja

- El Transmisor debe tomar en cuenta los segmentos en tránsito
- Luego el número de bytes que el Tx puede enviar es en general menor que el anunciado por la RevWindows.

¿Cuál es la función principal o propósito del control de flujo?

Impedir que el transmisor envíe más datos que los que puede almacenar el receptor.

Contenido de Capítulo 3

3.1 Servicios de la capa transporte

3.2 Transporte sin conexión: UDP

3.3 Transporte orientado a la conexión: TCP

- Estructura de un segmento
- Transferencia confiable de datos
- Control de flujo
- Gestión de la conexión

Administración de Conexión en TCP

Recordar: Transmisor y receptor TCP establecen una "conexión" antes de intercambiar segmentos de datos

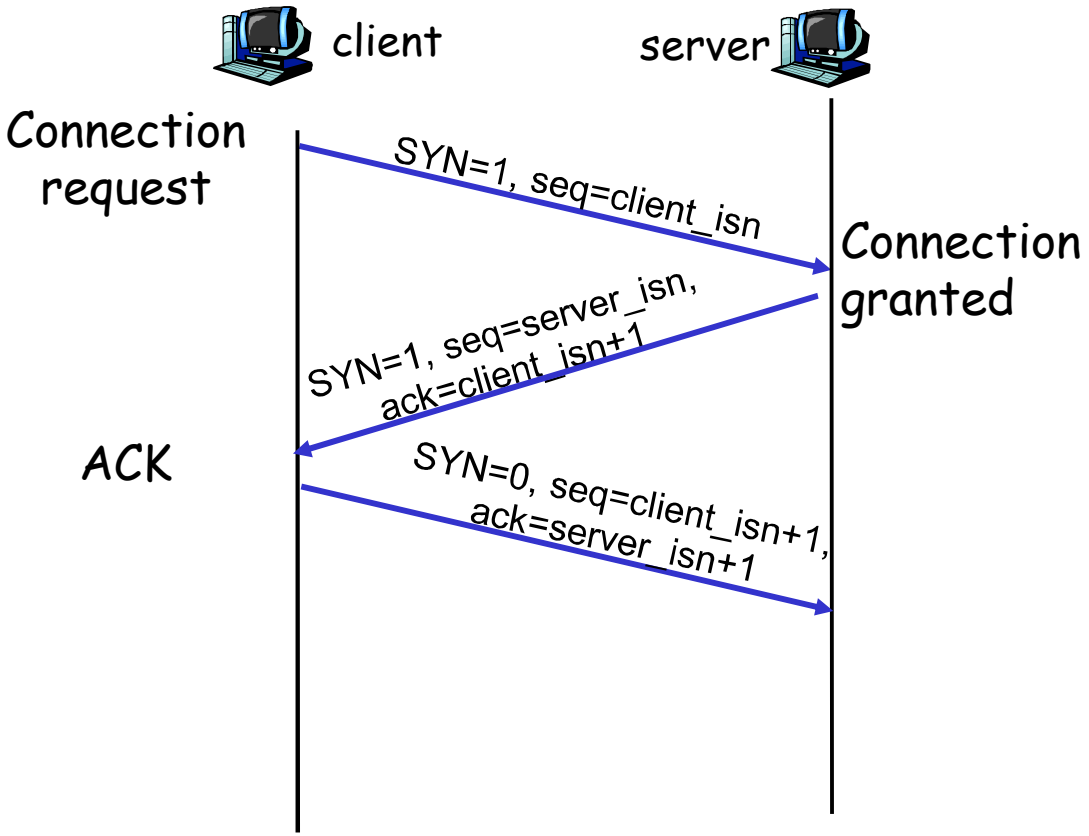
- TCP inicializa variables:
  - # de secuencia
  - buffers, información de control de flujo (e.g. RcvWindow)
- *cliente:* Iniciación de conexión
- *server:* contactado por cliente

Saludo de manos de tres vías (Three way handshake):

- Paso 1: host cliente envía segmento TCP SYN al servidor
  - Informa # secuencia inicial
  - Sin datos
- Paso 2: host servidor recibe SYN, responde con segmento SYN & ACK
  - Servidor ubica buffers
  - Informa su # secuencia inicial
- Paso 3: cliente recibe SYN & ACK, responde con segmento ACK, el cual podría contener datos.

Administración de Conexión en TCP (cont.)

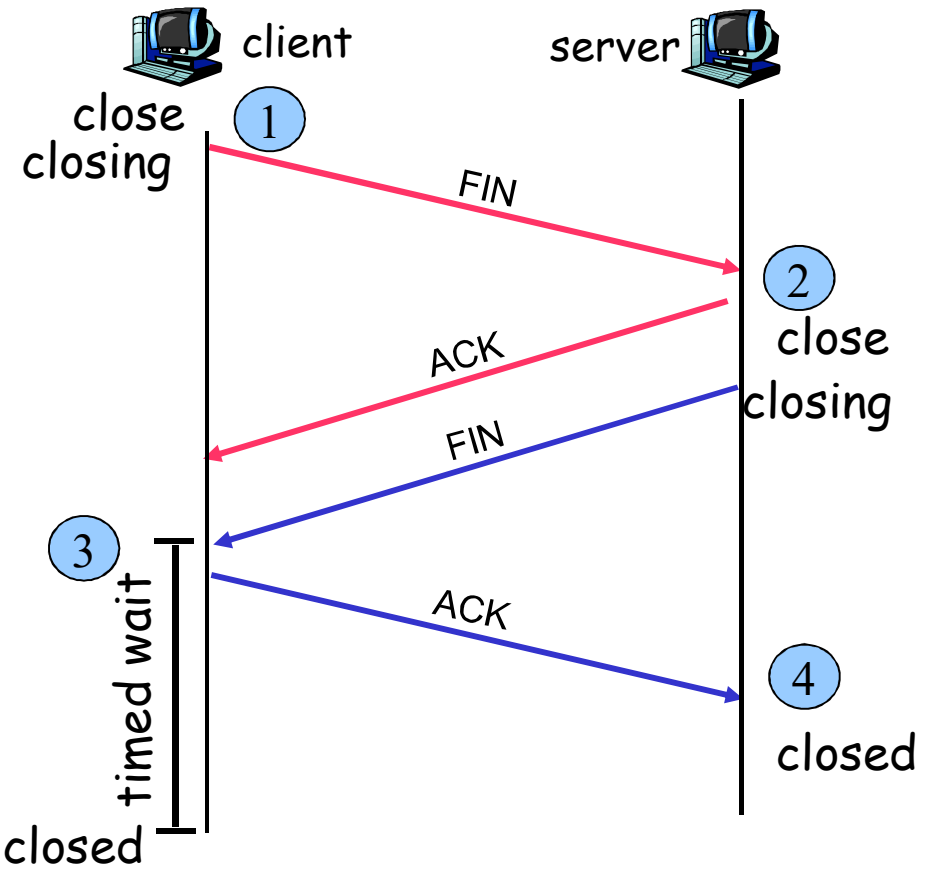
- Establecimiento de conexión



Administración de la conexión TCP (cont.)

Cerrando una conexión:

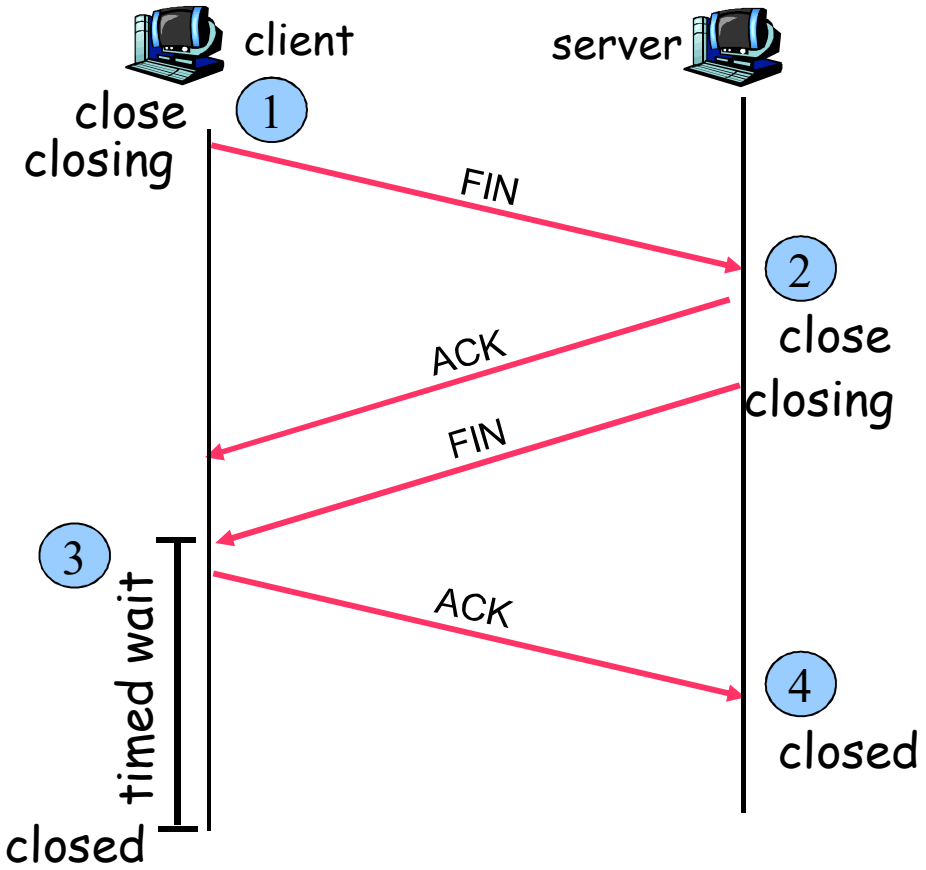
- Cliente cierra socket
- Puede ser iniciado por el servidor, aquí suponemos cierre iniciado por cliente.
- **Paso 1:** host cliente envía segmento TCP FIN al servidor
- **Paso 2:** servidor recibe FIN, responde con ACK. Ante un cierre de conexión de la aplicación y envía FIN.



Administración de la conexión TCP (cont.)

- **Paso 3:** cliente recibe FIN, responde con ACK.
  - Entra en "tiempo de espera" – responderá con ACK a FINs recibidos
- **Paso 4:** servidor, recibe ACK. Pasa a conexión cerrada.

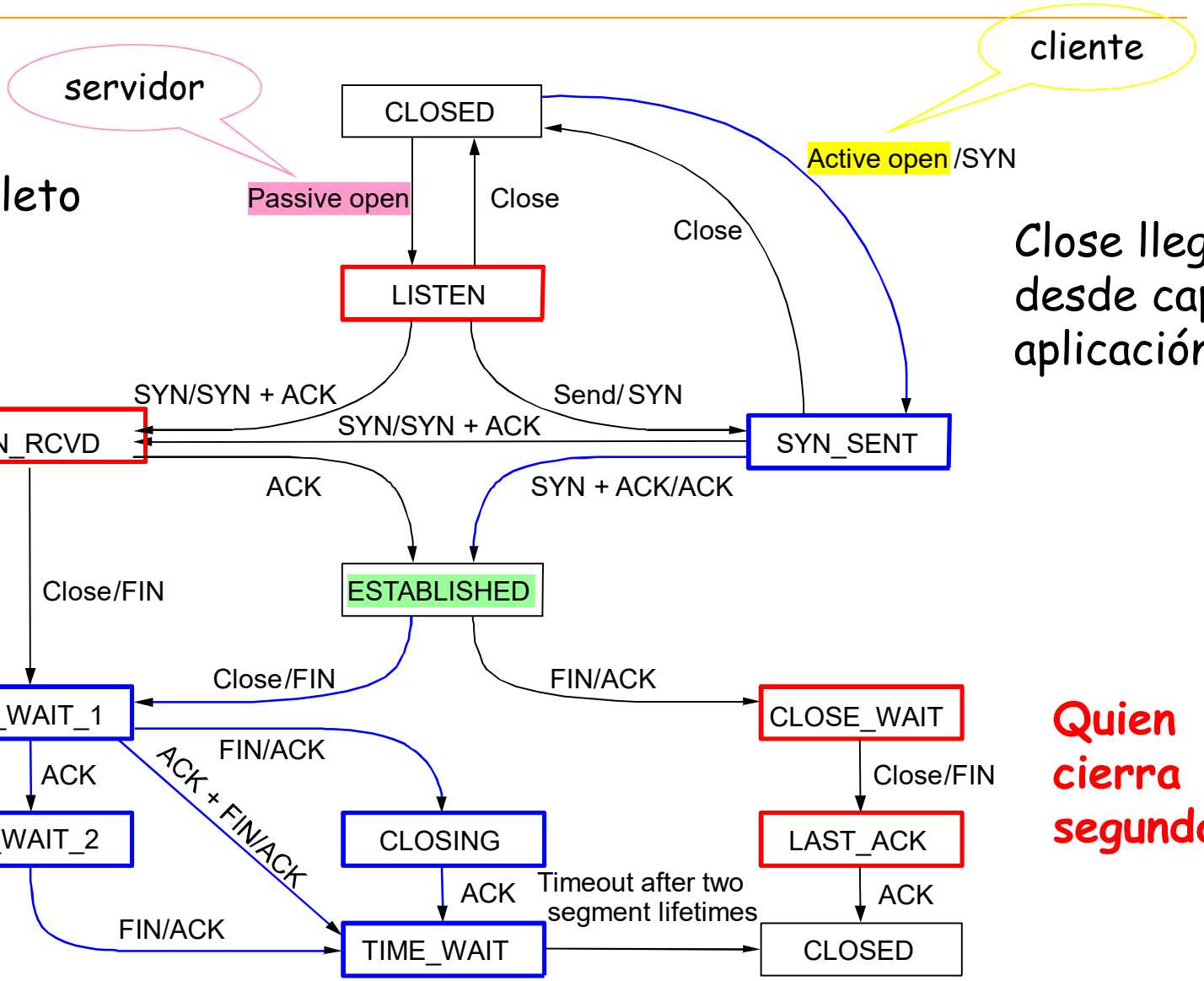
Nota: Con pequeña modificación se puede manejar FINs simultáneos.



Cualquiera de los dos puede iniciar el cierre

Administración de la Conexión TCP Completo

Diagrama de estados completo e incluyendo ambos casos



Close llega desde capa aplicación

Quien cierra primero

Quien cierra segundo